1 Backtesting System Design

Data layer. `src/data_loader.py` loads close-price data into a date × ticker DataFrame; both wide and long CSV layouts are auto-detected. Missing prices are forward-filled and tickers that remain entirely NaN are dropped. Daily simple returns $R_{i,t} = P_{i,t}/P_{i,t-1} - 1$ live in a parallel panel of identical shape.

Strategy contract. `src/strategies/base.py` defines `BaseStrategy.generate_weights(prices_until_t, returns_until_t, current_date)`, returning a target weight Series for the current day. Subclasses override that single method; the engine handles everything else. A shared `validate_weights` method clips negatives to zero, normalizes if the gross exceeds 1, and treats the residual as cash. Three reusable weighting helpers (`equal_weight`, `inverse_volatility_weight`, `rank_weight`) live in `src/portfolio.py`.

Engine. `src/backtester.py` iterates the trading calendar. At day t it slices prices/returns to rows `[: t]`, asks the strategy for w_t, validates, and *realizes those weights against* $R_{i,t+1}$*:*

$$R_{p,\,t+1} = \sum_i w_{i,t} R_{i,t+1} - \kappa \, \|w_t - w_{t-1}\|_1, \qquad \mathrm{NAV}_{t+1} = \mathrm{NAV}_t(1 + R_{p,\,t+1}).$$

The strategy never sees $R_{i,t+1}$ when choosing $w_{i,t}$; this is the structural no-lookahead guarantee. An optional `update_after_return` hook lets adaptive strategies (the UCB meta-strategy in §6) receive realized rewards, and is fired *after* $R_{p,t+1}$ is computed.

Metrics. `src/metrics.py` computes cumulative return, annualized return, annualized volatility, Sharpe (zero risk-free rate), maximum drawdown, Calmar, win rate, and average L1 turnover. All annualizations use $T = 252$.

Outputs. Each phase writes a fixed-schema set of artifacts. `outputs/tables/<phase>_metrics.csv` carries one row per strategy with columns `strategy, cumulative_return, annualized_return, annualized_volatility, sharpe_ratio, max_drawdown, calmar_ratio, win_rate, average_turnover`; `outputs/figures/<phase>_nav.png` and `<phase>_drawdown.png` are overlaid NAV and drawdown plots; and `outputs/logs/<phase>_log.json` contains the dataset bounds, strategy parameters, a verbatim copy of the metrics table, and any phase-specific bookkeeping (tuned grid params, bandit selection counts, bootstrap quantiles). Identical column names across CSVs make cross-phase comparison trivial.

Assumptions. (i) Weights execute at the recorded close; (ii) zero risk-free rate; (iii) long-only, no leverage; (iv) transaction cost defaults to 0 (the engine accepts a proportional κ for sensitivity); (v) no corporate-action handling, prices treated as adjusted; (vi) static universe equal to the loaded CSV (survivorship-biased).

Module map. `src/` (config, data_loader, backtester, metrics, plotting, portfolio, risk), `src/strategies/` (base, single_stock, benchmarks, cross_sectional, bandit), `experiments/` (one entry script per deliverable), `outputs/`, and `report/`.

System Architecture and Data Flow

The system separates concerns along a five-stage pipeline:

$$\text{Data} \rightarrow \text{Signal} \rightarrow \text{Portfolio} \rightarrow \text{Execution} \rightarrow \text{Evaluation}.$$

Data (`data_loader.py`) ingests close prices and emits two aligned panels: `prices` ($T \times N$) and `returns` = `prices.pct_change()`, both indexed by the trading calendar. **Signal** lives inside each strategy's `generate_weights` call — the cross-sectional or time-series statistic used to rank or gate names (e.g. trailing-return rank, ρ/σ score, market drawdown), consuming `prices[:t]` and `returns[:t]` only. **Portfolio construction** maps the signal to a target weight vector w_t; the helpers in `portfolio.py` (`equal_weight`, `inverse_volatility_weight`, `rank_weight`) take care of non-negativity, gross ≤ 1, and cash residual by construction. **Execution** is the engine itself: it calls `validate_weights`, charges $\kappa \, \|w_t - w_{t-1}\|_1$, and realizes w_t against $R_{i,t+1}$. **Evaluation** (`metrics.py`, `risk.py`, `plotting.py`) consumes the resulting NAV / daily-return series and produces metrics, drawdown charts, and risk diagnostics. The pipeline is strictly forward: evaluation cannot feed back into execution.

Extensibility and Strategy Integration

A new strategy requires implementing one method. The engine, metrics, plotting, portfolio helpers, and constraint enforcement are untouched.

```
class MyNewStrategy(BaseStrategy):
    name = "my_new_strategy"

    def generate_weights(self, prices_until_t, returns_until_t,
                         current_date):
        signal   = my_signal(prices_until_t, returns_until_t)  # (1) signal
        selected = signal.nlargest(self.k).index                # (2) select
        return inverse_volatility_weight(                       # (3) weight
            selected, vol_estimate, prices_until_t.columns
        )
```

The engine then runs `Backtester(prices).run(MyNewStrategy())` and writes the standard outputs without further glue. All eight strategies in this report — five single-stock, two new portfolio, plus the UCB meta-strategy — were added this way.

Acknowledged limitation. `generate_weights` merges signal construction with portfolio construction in a single method. A cleaner factoring would split a strategy into composable `Signal` and `PortfolioBuilder` pieces, so a top-K momentum signal could be paired with either equal-weight or inverse-vol weighting without writing a new strategy class. The current design accepts that duplication for the simplicity of a one-method contract.

2 Design Choices

Next-day return realization, not shifted weights. The strategy sees `prices[:t]` (close of t inclusive) and is paid by $R_{i,t+1}$. The alternative — pay w_t with $R_{i,t}$ — would force the strategy to choose with information through $t-1$, which conflates "observe the close" with "trade on the close". The chosen pattern matches how a real desk works: decide at the close, realize the move overnight to the next close.

Constraint enforcement in the base class. `validate_weights` clips negatives, normalizes if gross > 1, and asserts the invariants. Pushing this into the engine means a new strategy author cannot accidentally short or lever, and the engine never has to trust the subclass.

Positive-return gate on the new momentum strategies. Restricting selection to names with $\rho_{i,t} > 0$ converts top-K momentum into a long-or-cash rule for free: in a broad-market

sell-off the entire cross-section can be negative and the strategy parks in cash instead of loading on the "least bad" names.

K=3 with inverse-volatility weighting. A concentrated portfolio is dominated by idiosyncratic vol, so inverse-vol weighting matters more there than at K=10 where the spread is already diluted. The grid agreed with this intuition — the inverse-vol variant won at K=3 but only marginally at K=10. Sitting at the boundary (K=3, ℓ=90) is flagged in §5 as an overfitting risk; with this sample length the in-sample optimum clearly wants *more* concentration and *longer* memory than the grid offers, but K=1 is no longer a portfolio strategy.

Drawdown threshold −10% on a 120-day window. −5% triggers on routine intra-month volatility (too many false positives), −15% only fires after the worst is already past (too late). 120 trading days ≈ 6 months gives a smooth medium-term drawdown signal that does not whipsaw on single-week sell-offs. The grid confirmed −10% as the in-sample winner.

60/40 split rather than walk-forward. A single split is transparent and easy to audit, which is the right trade-off for a course project. Walk-forward retuning would reduce overfitting more, at the cost of an extra hyperparameter (refit frequency) and a much heavier compute bill — worth doing as a follow-up but not the headline protocol here.

3 Deliverable 3 – Single-Stock Strategy Comparison

Test five long-or-cash strategies on NVDA over the full 1,255-day sample: 20-day momentum; 20-day mean reversion (long after a −5% trailing return); 20/50 SMA crossover; volatility-filtered momentum (long iff trailing return > 0 AND 20-day vol < 3%); and 20-day z-score mean reversion (long iff $z_t < -1$). Each strategy is a single-stock specialization of the generic `BaseStrategy` — no engine code changes.

Table 1: Single-stock (NVDA) backtest metrics, full sample.

Strategy	Cum. Ret.	Ann. Ret.	Ann. Vol.	Sharpe	Max DD	Turnover
Momentum (20)	3.694	0.364	0.384	**0.997**	−0.498	0.095
Z-score mean reversion (20)	2.022	0.249	0.262	0.977	**−0.226**	0.110
Vol-filtered momentum	2.131	0.258	0.272	0.974	−0.364	0.076
MA crossover (20/50)	3.042	0.324	0.376	0.930	−0.636	**0.022**
Mean reversion (−5%)	1.838	0.233	0.298	0.851	−0.407	0.075

Interpretation. The top three Sharpes (momentum 0.997, z-score 0.977, vol-filtered 0.974) are within 0.02 of each other — well inside the noise band of a 5-year single-asset Sharpe estimate (the approximate standard error of an annualized Sharpe at N=1255 is ≈ 0.45; see §6). Momentum captures the most return (3.7×) but pays a −49.8% drawdown; z-score mean reversion spends most of the period in cash and gives back roughly half the return for less than half the drawdown, which is why it tops the Calmar ranking. MA crossover is the lowest-turnover rule by an order of magnitude.

4 Deliverable 4 – Portfolio-Level Backtest

Four constructions are tested over the full 101-stock pool. Two pin the selection rule to the top-10 names by trailing 30-day return and vary *only* the weighting scheme, isolating the uniform vs. inverse-vol comparison required by the rubric. The SMA crossover portfolio (equal-weighted)

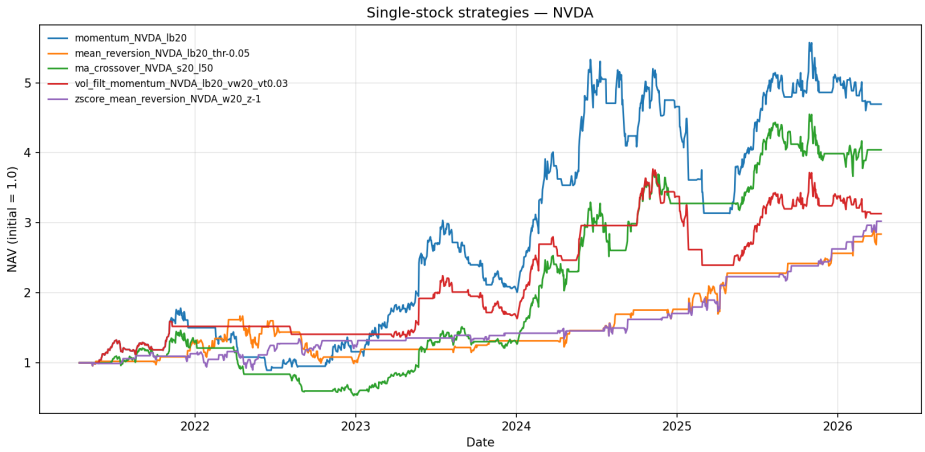

Figure 1: Single-stock NAV curves on NVDA.

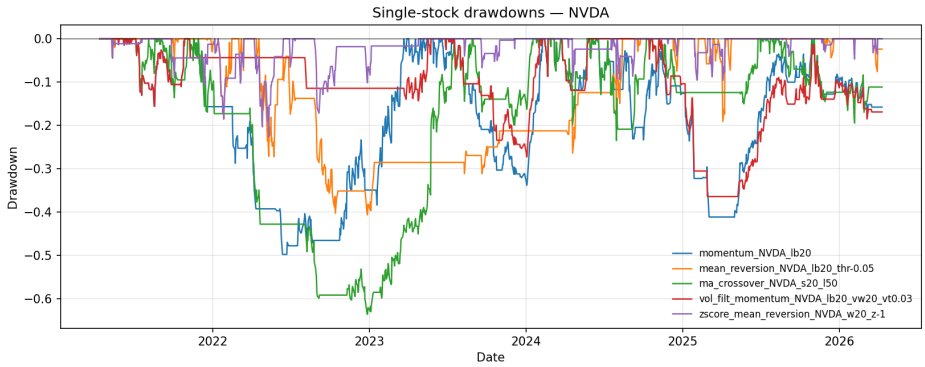

Figure 2: Single-stock drawdown profiles on NVDA.

and the equal-weight buy-and-hold of the entire pool are included as anchors. Inverse-volatility uses a 20-day rolling daily std.

Table 2: Portfolio-level backtest metrics, full sample.

Strategy	Wt.	Cum. Ret.	Ann. Vol.	Sharpe	Max DD	Turnover
Top-10 momentum (lb=30)	Unif	2.372	0.276	**1.022**	−0.430	0.328
Risk-adj top-10 momentum	IV	1.187	**0.181**	0.961	**−0.275**	0.443
SMA crossover (20/50)	Unif	1.964	0.357	0.752	−0.255	**0.082**
Equal-weight buy-and-hold	Unif	0.930	0.207	0.741	−0.310	0.014

Interpretation. On the same selection rule, uniform weighting edges inverse-vol on Sharpe (1.02 vs. 0.96) and decisively wins on cumulative return (2.37× vs. 1.19×); inverse-vol cuts volatility by about a third (27.6% → 18.1%) and the drawdown by roughly half (−43% → −27%). The reason uniform wins on Sharpe in this sample is regime-specific: the highest-momentum names in 2021–2026 are also the highest-volatility mega-cap tech names, so inverse-vol weighting under-allocates to exactly the winners. This is the empirical motivation for the Deliverable-5 strategies that *concentrate* into the strongest momentum names rather than dilute them via vol scaling.

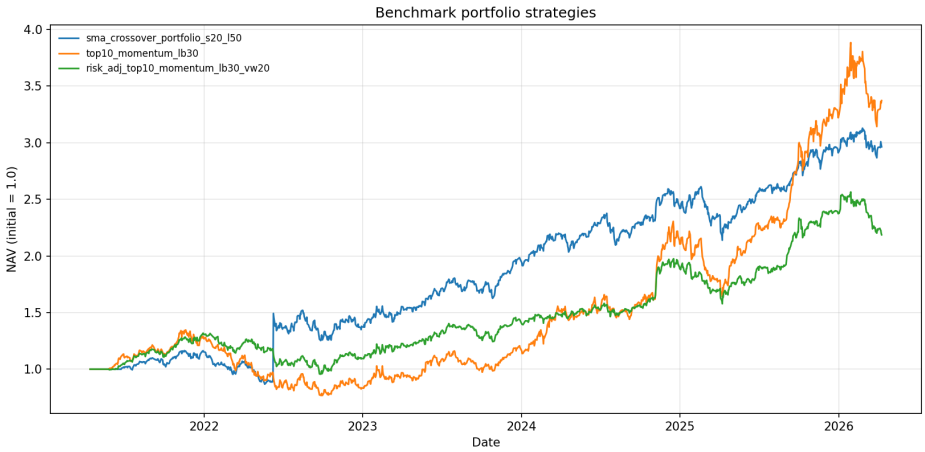

Figure 3: Portfolio NAV. Uniform top-10 momentum dominates on cumulative growth.

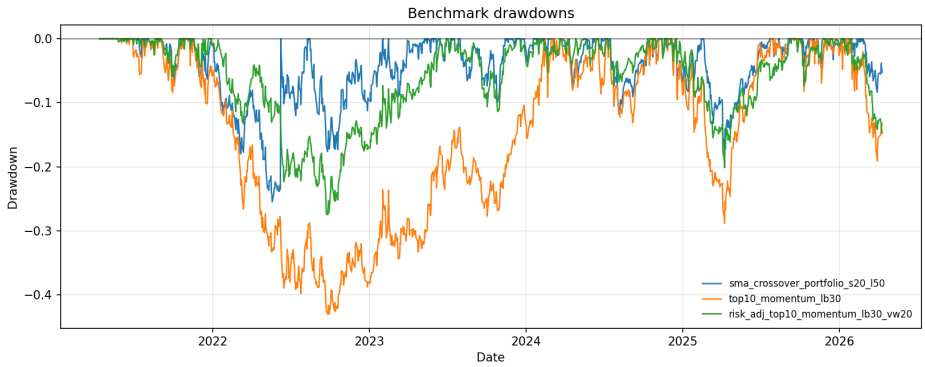

Figure 4: Portfolio drawdowns. Inverse-volatility weighting nearly halves the worst drawdown of top-10 momentum.

5 Deliverable 5 – Benchmarks vs. Two New Strategies

Benchmark 1 (SMA crossover). Equal-weight Nasdaq-100 stocks whose 20-day SMA exceeds the 50-day SMA; cash if none qualify. `SMACrossoverPortfolioStrategy(20, 50)`.

Benchmark 2 (Top-K momentum). Equal-weight the top 10 names by trailing 30-day return. `TopKMomentumStrategy(30, 10)`.

New strategy 1 – ConcentratedMomentum. Top-K by raw trailing ℓ-day return, restricted to names with strictly positive trailing return, weighted equal- or inverse-vol (chosen by the grid). Concentrating into a small K raises the loading on the strongest momentum signal; the positive-return gate parks the portfolio in cash during broad-market downturns. `ConcentratedMomentumStrategy(lookback, k, weighting, volatility_window)`.

New strategy 2 – MomentumWithDrawdownControl. Same top-K positive-return selection, equal-weighted; if the equal-weight market index has drawn down by more than $|\text{thr}|$ over the last `market_window` days, gross exposure is scaled to `defensive_exposure` (residual in cash). The drawdown signal uses only past returns. `MomentumWithDrawdownControlStrategy(lookback, k, drawdown_threshold, defensive_exposure, market_window)`.

Protocol. Tune each new strategy on the first 60% of dates (2021-04-13 → 2024-04-09) by maximizing in-sample Sharpe over a small grid; evaluate the tuned strategies and the two benchmarks on the held-out 40% test window (2024-04-10 → 2026-04-10, 502 days). Grid for ConcentratedMomentum: `lookback`$\in \{20, 30, 45, 60, 90\}$, $k \in \{3, 5, 7, 10, 15\}$, `weighting`$\in$\{equal, in-

verse_vol}, `vol_window`=20. Grid for MomentumWithDrawdownControl: `lookback`$\in \{20, 30, 45, 60, 90\}$, $k \in \{3, 5, 7, 10\}$, `drawdown_threshold`$\in \{-0.05, -0.10, -0.15\}$, `defensive_exposure`=0.5, `market_window`=12
Both grid winners selected $k = 3$ and $\ell = 90$ (Concentrated chose `inverse_vol`; DD-control chose `thr`$= -0.10$).

Table 3: Benchmarks vs. new strategies, held-out test window (2024-04-10 → 2026-04-10, 502 days).

Strategy	Cum. Ret.	Ann. Ret.	Ann. Vol.	Sharpe	Max DD
ConcentratedMom (k=3, ℓ=90, IV)	4.103	1.266	0.477	**1.951**	−0.397
MomDDCtrl (k=3, ℓ=90, thr=−10%)	3.843	1.207	0.465	**1.937**	−0.392
B2: Top-10 momentum (lb=30)	1.246	0.501	0.279	1.594	−0.288
B1: SMA crossover (20/50)	0.306	0.143	0.167	0.884	**−0.181**

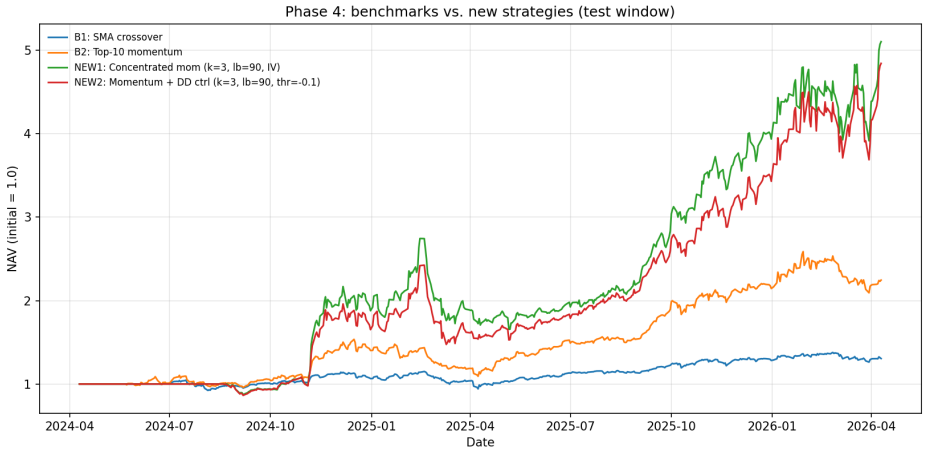

Figure 5: Test-window NAV. Both new strategies finish well above the top-K benchmark.

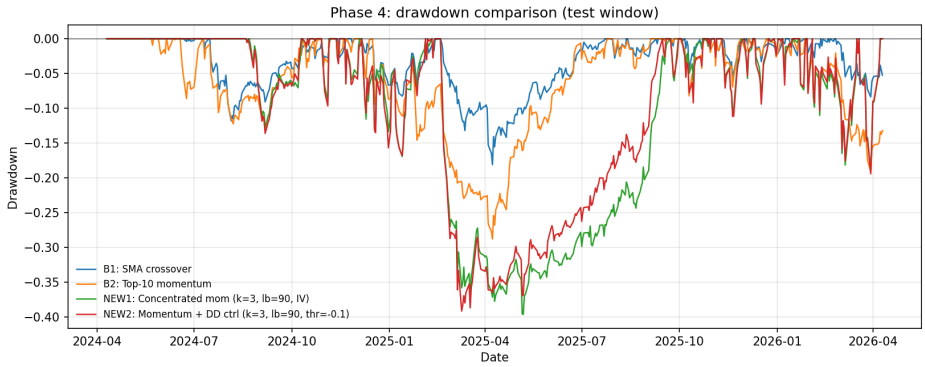

Figure 6: Test-window drawdowns. The new strategies pay for upside with deeper drawdowns ($\approx -40\%$) than top-K (-29%).

Result. Both new strategies beat both benchmarks on Sharpe. ConcentratedMomentum at 1.951 and MomDDCtrl at 1.937 clear the top-K benchmark (1.594) by 0.34–0.36 and the SMA benchmark (0.884) by more than a full unit. The verdict is recorded in `outputs/logs/new_strategies_log.js` with `beats_both_benchmarks: true` for both strategies.

Caveats. Three caveats temper the read. (i) The Sharpe gain comes mostly from amplified *return* ($\approx 2.5\times$ top-K's ann. ret.) and only partly from a worse *volatility* trade ($\approx 1.7\times$); MDD is correspondingly deeper (-40% vs. -29%). Higher Sharpe is not lower risk. (ii) Both grid winners sit at the boundary of the search ($k = 3$ smallest, $\ell = 90$ longest), a structural

overfitting flag — the true optimum may lie outside the grid. (iii) On the test window the standard error of an annualized Sharpe of 1.95 with $N = 502$ is $\sqrt{(1 + \frac{1}{2}\widehat{\mathrm{SR}}^2)/N} \cdot \sqrt{T} \approx 0.71$, so the 95% CI on the new strategies overlaps with top-K's CI; beating the top-K benchmark by 0.34 Sharpe is real evidence but not statistically overwhelming. The strategies should be re-validated under walk-forward retuning, realistic transaction costs, and market regimes other than the 2024–2026 AI rally before being treated as production-ready.

6 Extensions: UCB Meta-Strategy and Risk Diagnostics

UCB-1 over four arms. `UCBMetaStrategy(arms, c=1.0)` selects each day by $\mathrm{UCB}_k(t) = \overline{r}_k + c\sqrt{\ln T(t)/n_k(t)}$ across SMA crossover, top-10 momentum, risk-adjusted momentum (ℓ=60), and low-volatility momentum (ℓ=60); the chosen arm's weights are forwarded and `update_after_return` credits that arm's reward after the realized $R_{p,t+1}$. Selection counts on the full sample are nearly uniform (315/315/313/311 of 1,254 trials) because the per-day reward differences are sub-basis-point and the exploration bonus dominates.

Table 4: UCB meta-strategy and arms, full sample.

Strategy	Cum. Ret.	Ann. Vol.	Sharpe	Max DD	Calmar
Top-10 momentum (lb=30)	2.372	0.276	**1.022**	−0.430	0.642
Risk-adj momentum (ℓ=60)	0.998	0.169	0.908	**−0.200**	0.747
UCB meta (c=1, n=4)	2.079	0.361	0.768	−0.234	**1.084**
SMA crossover (20/50)	1.964	0.357	0.752	−0.255	0.957
Low-vol momentum (ℓ=60)	0.399	0.149	0.528	−0.215	0.325

UCB does not improve Sharpe over the best individual arm but cuts max drawdown by $\sim$ 20pp vs. top-K and posts the best Calmar in the table. The cost is $\sim 4\times$ higher turnover; under any realistic transaction cost the Sharpe gap would widen against UCB.

Tail risk and Sharpe uncertainty. `src/risk.py` computes empirical 5%-VaR / 5%-CVaR, an i.i.d. bootstrap of NAV paths, and an approximate normal CI for the annualized Sharpe under i.i.d.-Gaussian returns, $\mathrm{SE}(\widehat{\mathrm{SR}}) \approx \sqrt{(1 + \frac{1}{2}\widehat{\mathrm{SR}}^2)/N} \cdot \sqrt{T}$. This is not a full Lo (2002) HAC adjustment; the intervals widen if returns are autocorrelated or fat-tailed. Top-10 momentum has the worst tail (CVaR −3.82%, MDD −43%); risk-adjusted momentum (ℓ=60) has the cleanest tail (CVaR −2.51%, MDD −20%). On the full sample only top-10 momentum's 95% Sharpe CI ($[0.14, 1.90]$) sits strictly above zero; the other arms' CIs all include zero, so on this sample length differences smaller than $\sim$ 0.5 Sharpe are statistically indistinguishable from noise.

7 Limitations and Reproducibility

Limitations. Daily-close fillability with no spread or slippage; zero default transaction cost; survivorship bias in the constituent panel; single 60/40 split with boundary-parameter selection in Deliverable 5; a single dominant macro regime (post-COVID rates → AI mega-cap rally) in the 5-year window; Sharpe CIs assume i.i.d. returns. The next two subsections take the two most material limitations — transaction costs and statistical robustness — in turn.

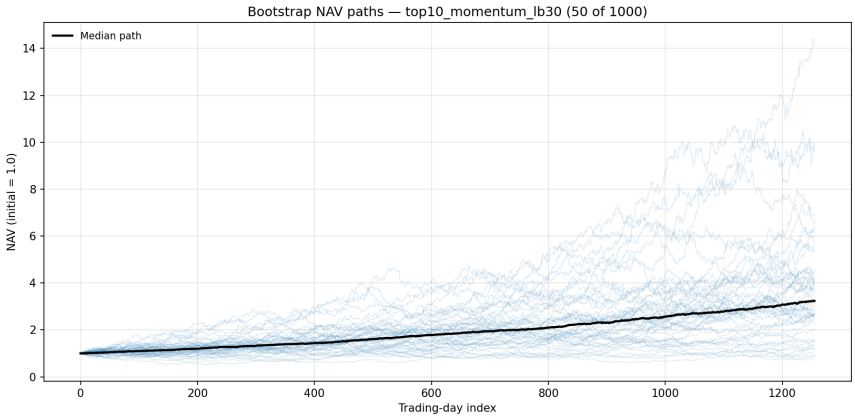

Figure 7: Bootstrap NAV paths for top-10 momentum. With identical mean and variance, the 5th–95th percentile spread of terminal NAV is roughly 8× (1.18 → 9.18); much of any realized track record is sequence luck.

Transaction Costs and Market Frictions

The default $\kappa = 0$ keeps cross-strategy comparisons free of an ad-hoc cost assumption, but it is unrealistic. A plausible proportional cost on the close for liquid US equities is $\kappa \in [5, 15]$ bps per side once spread, exchange fees, and small-size impact are accounted for. The first-order Sharpe drag is $\Delta\text{Sharpe} \approx -\sqrt{T}\,\kappa\,\overline{\text{TO}}/\sigma_p$, which scales linearly with average daily turnover $\overline{\text{TO}}$.

Strategy	**Avg. daily TO**	**Vulnerability**
Single-stock MA crossover	0.02	negligible
SMA crossover portfolio	0.07–0.08	low
MomDDCtrl (k=3, ℓ=90)	0.17	moderate
ConcentratedMomentum (k=3, ℓ=90)	0.24	moderate
Top-K momentum (lb=30)	0.30–0.33	moderate
Risk-adj top-K momentum	0.44	high
UCB meta-strategy	1.40	severe

At $\kappa = 10$ bps the UCB meta-strategy bleeds $252 \cdot 0.0010 \cdot 1.40 \approx 35\%$ in annualized return — enough to push its Sharpe well below zero. Top-K momentum loses $\sim 7.5\%$ annualized (≈ 0.27 Sharpe given its 27.6% volatility). SMA crossover loses only $\sim 2\%$. The ranking compresses materially under realistic costs and the low-turnover trend-following arms become more competitive relative to the high-Sharpe momentum strategies. The engine already accepts κ as a constructor argument, so a $\kappa \in \{0, 5, 10, 20\}$ bps sensitivity sweep slots in without code changes. A per-name fixed cost and a volume-aware slippage model are the natural next refinements (the latter currently blocked by the absence of a volume column in the dataset).

Robustness and Statistical Considerations

Parameter sensitivity. Both Deliverable-5 winners selected *boundary* parameters ($k = 3$ smallest, $\ell = 90$ longest). Boundary selections are a structural overfitting flag — the in-sample objective is still climbing at the edge of the grid, so the true optimum may lie outside what was searched. A natural diagnostic is to widen the grid (e.g. $k \in \{1, 2, 3, 5, 7\}$, $\ell \in \{60, 90, 120, 180\}$) and verify the optimum moves interior; if it does not, the signal genuinely prefers the most concentrated, longest-memory configuration available, and the limit is the grid not the strategy.

Single 60/40 split. One train/test cut gives one realization of out-of-sample performance,

conditional on one test regime. The 2024-04 → 2026-04 test window is dominated by an AI mega-cap rally — precisely the regime that favors concentrated long-only momentum — so the verdict is partly a statement about the regime, not solely about the strategy. Three more defensible protocols:

- **Walk-forward retuning.** Re-fit every Δ months on a rolling window of the last W years; report a sequence of out-of-sample Sharpes rather than one.
- **k-fold CV across years.** Hold out one calendar year at a time; aggregate the five out-of-sample Sharpes.
- **Block bootstrap.** Resample contiguous blocks of returns to preserve volatility clustering and re-estimate Sharpe.

Sharpe estimation uncertainty. The Sharpe ratio is an estimate, not a known quantity. The i.i.d.-Gaussian closed-form $\mathrm{SE}(\widehat{\mathrm{SR}}) \approx \sqrt{(1+\frac{1}{2}\widehat{\mathrm{SR}}^2)/N} \cdot \sqrt{T}$ gives ≈ 0.45 at $N = 1{,}255$ and ≈ 0.71 at $N = 502$. Two practical consequences: (i) Sharpe differences smaller than ≈ 0.5 on the full sample (or ≈ 0.7 on the test window) are statistically indistinguishable from noise; (ii) the i.i.d. formula is a *lower bound* on the true uncertainty, since volatility clustering and autocorrelation in daily returns inflate the variance of the Sharpe estimator. A Lo (2002) HAC correction is the natural next refinement.

Reproducibility. Place the dataset at `data/nasdaq100_daily_5y.csv` and run from the project root:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python experiments/run_all.py            # engine smoke test
python experiments/run_single_stock.py   # D3
python experiments/run_benchmarks.py     # D4
python experiments/run_new_strategies.py # D5
python experiments/run_ucb_strategy.py   # extension
python experiments/run_risk_analysis.py  # extension
```

(On macOS, set `MPLBACKEND=Agg` before running plot-producing scripts to avoid backend autodetection.) Each script writes a CSV metrics table, NAV/drawdown PNGs, and a JSON log to `outputs/tables/`, `outputs/figures/`, `outputs/logs/`. The Deliverable-5 verdict is computed in `outputs/logs/new_strategies_log.json`.

Run identification and config snapshots. Each `<phase>_log.json` already embeds dataset bounds, strategy parameters, and the metrics table, so a log file is sufficient to identify a run. A natural extension is to capture (i) the git commit SHA at run time, (ii) the random seed wherever stochastic methods are used (currently fixed at `42` in `src/risk.py` for the bootstrap and stress test), and (iii) the Python / pandas / numpy versions, all under a single `run_id` field. This would let a third party re-execute any historical experiment unambiguously, even across code revisions.